Rota das Coisas:
Sistema de ABS Distribuído Baseado em Protocolos Híbridos

Enzo Cauã da S. Barbosa

[1]Departamento de Tecnologia – Universidade Estadual de Feira de Santana (UEFS)
Feira de Santana – BA – Brasil

caua7uefs@gmail.com

Abstract. *Internet of Things enables the integration of sensors and actuators in industrial systems. Point-to-point communication creates high coupling and overhead in resource-constrained devices, causing packet loss. This paper presents a distributed Anti-lock Braking System (ABS) using a centralized integration service for data decoupling. The solution was developed in Java with hybrid Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) protocols, using multithreading techniquess. Tests validated the service discovery efficiency and the watchdog mechanism's robustness against connection failures.*

Resumo. *A Internet das Coisas possibilita a integração de sensores e atuadores em sistemas industriais. A comunicação ponto a ponto gera alto acoplamento e sobrecarga em dispositivos de recursos limitados, resultando em perda de pacotes. Este trabalho apresenta um sistema de frenagem antibloqueio (ABS) distribuído com um serviço de integração centralizado para desacoplamento de dados. A solução foi desenvolvida em Java com protocolos híbridos Transmission Control Protocol (TCP) e User Datagram Protocol (UDP), utilizando tecnicas de multithreading. Os testes validaram a eficiência da descoberta automática e a robustez do watchdog frente a falhas de conexão.*

1. Introdução

A evolução da Internet das Coisas (IoT) tem possibilitado a integração de componentes físicos ao ambiente digital [Atzori et al. 2010, de Oliveira et al. 2014]. Contudo, arquiteturas do tipo ponto a ponto (*peer-to-peer*) exigem que dispositivos sensores gerenciem múltiplas conexões simultâneas, o que pode comprometer sua eficiência em sistemas embarcados [de Oliveira et al. 2014]. Dispositivos físicos frequentemente apresentam limitações de memória e capacidade de processamento, tornando esse modelo ineficiente e suscetível a falhas operacionais.

Neste contexto, este projeto propõe um sistema de frenagem antibloqueio (*Anti-lock Braking System* – ABS) distribuído, composto por módulos independentes. Um servidor central atua como elemento integrador, sendo responsável pelo gerenciamento do fluxo de telemetria e dos comandos de controle [Coulouris et al. 2005].

A implementação foi realizada na linguagem Java, utilizando a API de *sockets* nativa [Oracle 2024]. A comunicação do sistema adota uma abordagem híbrida, empregando o *User Datagram Protocol* (UDP) para transmissão de dados de telemetria, devido

à sua baixa latência, e o *Transmission Control Protocol* (TCP) para o envio de comandos críticos, em função de sua confiabilidade [Tanenbaum and Wetherall 2011].

A solução desenvolvida apresentou baixa latência na atualização dos dados e elevada confiabilidade na execução dos comandos de frenagem. O restante deste relatório está organizado da seguinte forma: inicialmente, apresenta-se a fundamentação teórica; em seguida, a metodologia adotada; posteriormente, a discussão dos resultados; e, por fim, as conclusões do trabalho.

2. Fundamentação Teórica

Esta seção apresenta os principais conceitos teóricos que fundamentam o desenvolvimento do sistema proposto, abrangendo aspectos de redes de computadores, concorrência, sistemas distribuídos e controle aplicado.

2.1. Conectividade e Protocolos de Rede

A comunicação entre os módulos do sistema é baseada no modelo de arquitetura cliente-servidor, no qual um servidor central é responsável pelo processamento e gerenciamento das informações, enquanto os clientes atuam como produtores ou consumidores de dados [Coulouris et al. 2005]. No contexto deste projeto, sensores e atuadores se comportam como clientes, enquanto o servidor central coordena a lógica de controle.

A comunicação em rede é implementada por meio da API de *sockets*, que fornece uma interface de programação para troca de dados entre processos distribuídos. Essa abordagem permite o envio e recebimento de mensagens através de protocolos de transporte.

O UDP é um protocolo de comunicação não orientado à conexão, caracterizado por baixa latência e ausência de garantias de entrega. No sistema proposto, o UDP é utilizado para transmissão de telemetria dos sensores, onde a atualização rápida dos dados é mais relevante do que a confiabilidade absoluta.

Por outro lado, o TCP é orientado à conexão e garante a entrega ordenada e confiável dos dados. Essa característica é essencial para o envio de comandos críticos de frenagem, assegurando que instruções do servidor sejam corretamente recebidas pelos atuadores.

Adicionalmente, o mecanismo de *broadcast* é utilizado para envio de mensagens a todos os dispositivos de uma sub-rede, permitindo a implementação de técnicas de *service discovery*, nas quais os clientes podem localizar dinamicamente o servidor sem configuração manual de endereço IP.

2.2. Concorrência e Programação Paralela

O servidor do sistema deve ser capaz de lidar simultaneamente com múltiplos sensores e atuadores. Para isso, utiliza-se o conceito de *threads*, que representam unidades independentes de execução dentro de um processo. A utilização de múltiplas *threads* caracteriza o modelo de *multithreading*, permitindo o processamento concorrente de requisições.

Entretanto, a execução concorrente introduz desafios relacionados à consistência dos dados. Mecanismos de sincronização são empregados para garantir *thread-safety*, evitando condições de corrida (*race conditions*) [Coulouris et al. 2005]. Estruturas como

ConcurrentHashMap e mecanismos de coordenação como *CountDownLatch* são exemplos utilizados para esse fim.

Além disso, o uso de variáveis declaradas como *volatile* garante a visibilidade das alterações entre diferentes *threads*, assegurando que mudanças de estado sejam imediatamente propagadas no sistema [Oracle 2024].

2.3. Sistemas Distribuídos e Internet das Coisas

O sistema proposto está inserido no contexto de sistemas distribuídos, nos quais múltiplos componentes independentes cooperam para atingir um objetivo comum [Coulouris et al. 2005]. Nesse cenário, o desacoplamento entre os módulos é fundamental para aumentar a escalabilidade e a robustez.

A transição de uma arquitetura ponto a ponto para um modelo intermediado por um servidor central reduz o acoplamento entre sensores e atuadores, facilitando a manutenção e evolução do sistema.

No contexto da Internet das Coisas (IoT), sensores são responsáveis pela coleta de dados do ambiente, enquanto atuadores executam ações com base nesses dados [Atzori et al. 2010, de Oliveira et al. 2014]. No sistema ABS desenvolvido, sensores monitoram variáveis como velocidade das rodas, enquanto atuadores executam comandos de frenagem.

Outro aspecto crítico é o impacto da latência e do *jitter* na comunicação. Em sistemas de controle em tempo real, variações no tempo de entrega das mensagens podem comprometer a estabilidade e a segurança do sistema [Tanenbaum and Wetherall 2011]. Por isso, a escolha adequada dos protocolos de comunicação é essencial.

2.4. Lógica de Controle e Segurança

O sistema de frenagem ABS baseia-se no monitoramento do deslizamento das rodas, comparando a velocidade individual com uma referência para evitar o travamento. Essa lógica permite manter a aderência do veículo ao solo durante frenagens bruscas.

Para garantir a robustez do sistema, utiliza-se o mecanismo de *watchdog*, responsável por detectar falhas ou inatividade de componentes e acionar medidas de segurança (*fail-safe*), como a interrupção controlada do sistema [Coulouris et al. 2005].

Adicionalmente, a normalização de dados é aplicada para padronizar as mensagens provenientes de diferentes sensores, garantindo que o servidor consiga interpretá-las de maneira consistente, independentemente da origem.

3. Metodologia e Implementação

A solução proposta baseia-se em uma arquitetura modular distribuída, com o objetivo de promover o desacoplamento entre os componentes responsáveis pela coleta, processamento e atuação dos dados. O sistema foi desenvolvido na linguagem Java, utilizando a API de *sockets* para comunicação em rede e o Apache Maven para gerenciamento de dependências e padronização do ambiente de execução. O sistema atende diretamente aos requisitos do problema ao separar o tráfego de telemetria e controle, utilizando UDP para dados contínuos e TCP para comandos críticos, conforme especificado no cenário proposto.

3.1. Arquitetura do Sistema

O sistema é composto por quatro módulos independentes: sensores, atuadores, servidor central e painel do usuário. Essa divisão permite a separação de responsabilidades e reduz o acoplamento entre os componentes, uma vez que toda a lógica de controle e gerenciamento de conexões é centralizada no servidor. A arquitetura geral do sistema pode ser observada na Figura 1,

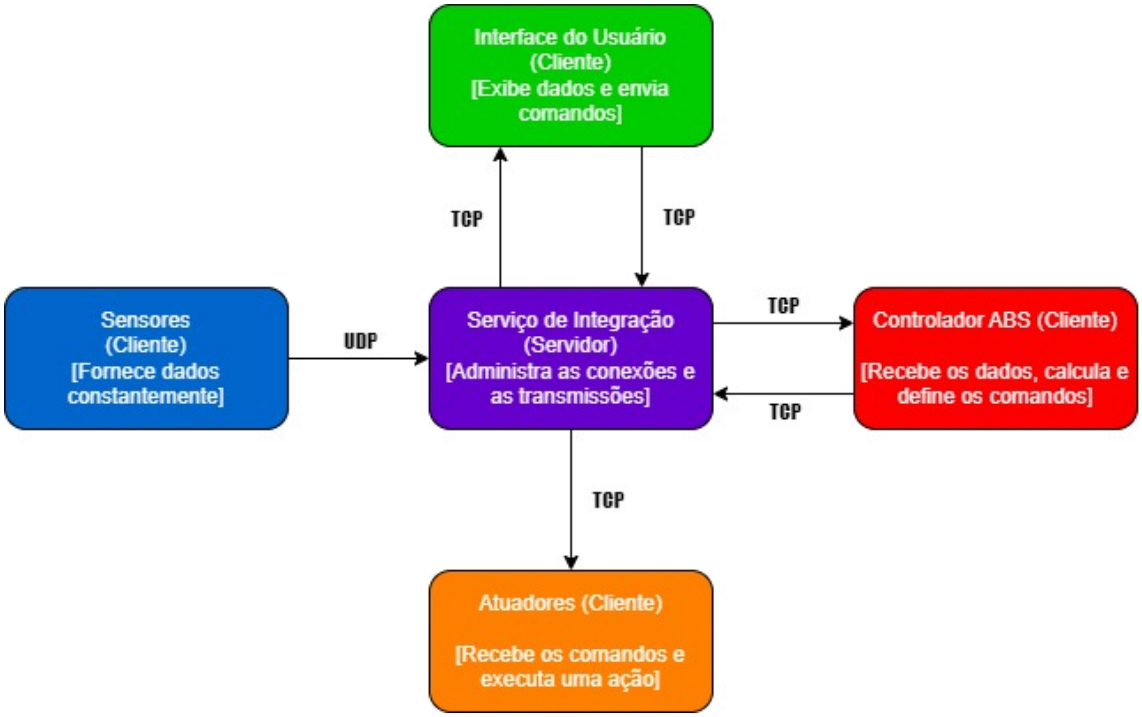

Figura 1. Arquitetura do sistema ABS distribuído

Os sensores são responsáveis pela coleta de dados de velocidade, enquanto os atuadores executam os comandos de frenagem. O painel do usuário permite a visualização do estado do sistema e o envio de comandos manuais. Cada módulo opera de forma independente e utiliza múltiplas *threads* para permitir processamento paralelo e comunicação simultânea.

3.2. Interface do Sistema

O sistema desenvolvido possui interfaces gráficas distintas para sensores, atuadores, servidor central e painel do usuário, responsáveis pela visualização e controle do estado do sistema.

A interface dos sensores permite o acompanhamento das velocidades geradas em tempo real. A interface dos atuadores apresenta os valores após o processamento da lógica de controle, juntamente com as porcentagens de atuação.

O painel do usuário possibilita a visualização das conexões ativas e a intervenção manual na lógica de frenagem. Já a interface do servidor central exibe o estado dos dispositivos conectados.

As interfaces desenvolvidas podem ser observadas no Apêndice A.

3.3. Descoberta de Serviço (Service Discovery)

A descoberta do servidor na rede é realizada dinamicamente por meio de mensagens de *broadcast* utilizando o protocolo UDP na porta 65000. O módulo responsável, denominado `GestorIP`, permanece em escuta contínua, respondendo às requisições dos clientes com o endereço IP do servidor ativo. Esse mecanismo elimina a necessidade de configuração manual de endereços, tornando o sistema mais flexível e adaptável a diferentes ambientes de execução.

3.4. Protocolos e Comunicação Híbrida

A comunicação entre os módulos foi projetada utilizando uma abordagem híbrida, combinando os protocolos UDP e TCP de acordo com os requisitos de cada tipo de dado.

A transmissão de telemetria dos sensores é realizada via UDP na porta 55000, priorizando baixa latência. Considerando a alta frequência de envio de dados, eventuais perdas de pacotes não comprometem significativamente o funcionamento do sistema.

Por outro lado, a comunicação de controle e sincronização de estados é realizada via TCP na porta 60000. O servidor estabelece conexões persistentes com atuadores e o painel do usuário, garantindo a entrega confiável e ordenada dos comandos de frenagem.

3.5. Processamento Central e Lógica de ABS

O processamento central é realizado no servidor, responsável por agregar as leituras recebidas dos sensores e aplicar a lógica de controle do sistema ABS. A classe `ServidorUDP` é utilizada para receber e organizar os dados provenientes das diferentes rodas.

A classe `LogicaABS` realiza a normalização das identificações dos sensores e calcula a velocidade de referência (v_{ref}), utilizada como base para a tomada de decisão. A intensidade da frenagem é definida a partir da diferença entre a velocidade individual de cada roda e essa referência.

Adicionalmente, o sistema permite a intervenção manual por meio do painel do usuário. Quando um comando manual é emitido, este possui prioridade sobre a lógica automática, garantindo maior controle ao operador. As decisões finais são enviadas aos atuadores por meio de conexões TCP.

3.6. Mecanismos de Robustez e Sincronização

Para aumentar a confiabilidade do sistema, foi implementado um mecanismo de *watchdog* responsável por monitorar a atividade dos sensores. Caso um dispositivo permaneça inativo por mais de três segundos, ele é removido da lista de sensores ativos, e um comando de segurança ("RESET") é enviado ao atuador correspondente. Esse mecanismo atua como *fail-safe*, evitando comportamentos inadequados em caso de falhas de comunicação.

A sincronização entre as *threads* de comunicação e a interface gráfica é realizada utilizando o mecanismo `CountDownLatch`. Essa abordagem garante que os componentes de rede sejam inicializados apenas após o carregamento completo da interface JavaFX, evitando inconsistências e exceções em tempo de execução.

3.7. Virtualização e Infraestrutura

A utilização de técnicas de virtualização por meio de contêineres, como o Docker, é amplamente empregada para garantir portabilidade e reprodutibilidade em sistemas distribuídos [Inc. 2025]. No entanto, durante o desenvolvimento deste projeto, verificou-se uma limitação prática relacionada ao uso da biblioteca JavaFX, que depende de recursos gráficos do sistema hospedeiro.

Ambientes de contêiner, por padrão, não possuem acesso direto à GPU ou a subsistemas gráficos, o que dificulta a execução de aplicações com interface gráfica sem configurações adicionais, como *GPU passthrough*. Alternativas como o uso de *X11 forwarding*, que permite o redirecionamento da interface gráfica para o sistema hospedeiro, ou a utilização de servidores gráficos remotos baseados em *Virtual Network Computing* (VNC), foram consideradas. Contudo, essas abordagens introduzem maior complexidade de configuração, dependência do ambiente de execução e possíveis impactos de desempenho e latência.

Essas limitações foram identificadas em estágio avançado do desenvolvimento, inviabilizando a adoção do Docker dentro do prazo estabelecido. Dessa forma, optou-se por não utilizar conteinerização na implementação final do sistema.

Como alternativa, utilizou-se o Apache Maven como ferramenta de gerenciamento de dependências e automação de build, garantindo maior padronização e consistência entre ambientes de desenvolvimento. O uso do Maven permite a resolução automática das bibliotecas necessárias, reduzindo inconsistências e facilitando a compilação e execução do sistema [Oracle 2024].

4. Resultados e Discussões

A avaliação da solução foi conduzida em um ambiente distribuído, simulando a interação entre múltiplos sensores, atuadores e o servidor central. Os experimentos tiveram como foco a eficiência da comunicação híbrida, a confiabilidade dos comandos críticos e a robustez do sistema frente a falhas.

4.1. Eficiência da Comunicação e Desacoplamento

A utilização do protocolo UDP para a transmissão de telemetria permitiu uma taxa de atualização estável de aproximadamente 500 ms por sensor, sem sobrecarga significativa da rede. Em cenários de maior volume de tráfego, observou-se uma taxa de perda de pacotes inferior a 2%.

Para fins de validação experimental e melhor observabilidade do comportamento do sistema, foram realizados testes adicionais com intervalos de envio de dados ajustados para 2000 ms. Essa configuração permitiu a análise visual detalhada das atualizações no painel do usuário, facilitando a verificação do correto funcionamento da lógica de processamento e exibição dos dados.

Também foram conduzidos testes com frequências mais elevadas de atualização (intervalos menores), representando cenários mais próximos de aplicações reais. No entanto, devido à alta velocidade de atualização dos dados, tornou-se inviável a verificação completa apenas por inspeção visual da interface. Ainda assim, o sistema manteve comportamento estável, sem indícios de falhas na comunicação ou inconsistências aparentes.

Esse comportamento está alinhado com as características do protocolo UDP, no qual não há garantia de entrega, porém a baixa latência compensa eventuais perdas. No contexto do sistema desenvolvido, a perda de pacotes não comprometeu a lógica de controle, uma vez que a alta frequência de envio permite rápida atualização do estado do sistema.

Além disso, o desacoplamento promovido pela arquitetura cliente-servidor foi validado experimentalmente. Os sensores passaram a transmitir dados para um único ponto de agregação (servidor UDP), eliminando a necessidade de múltiplas conexões diretas com diferentes consumidores. Essa abordagem reduziu a complexidade de gerenciamento de conexões e aumentou a escalabilidade do sistema.

4.2. Confiabilidade dos Comandos Críticos

Os comandos de controle e frenagem, transmitidos por meio do protocolo TCP, apresentaram entrega confiável e ordenada em todos os testes realizados, sem ocorrência de perdas.

A latência média observada para a execução de comandos de frenagem foi de aproximadamente 15 ms, valor compatível com os requisitos de sistemas de controle em tempo quase real. Esse resultado demonstra que a escolha de um modelo híbrido de comunicação foi adequada, combinando baixa latência para telemetria e alta confiabilidade para comandos críticos.

Adicionalmente, foi verificada a correta priorização de comandos manuais enviados pelo painel do usuário. Em todos os cenários testados, a intervenção manual sobrepôs a lógica automática do sistema ABS, garantindo previsibilidade e controle em situações críticas.

4.3. Análise de Falhas e Limitações

Durante os experimentos, foram identificadas limitações relevantes associadas à natureza distribuída do sistema. Uma falha abrupta de sensores resultava na manutenção de valores desatualizados no servidor, caracterizando uma condição de leitura inconsistente.

Para mitigar esse problema, o mecanismo de *watchdog* mostrou-se essencial, permitindo a detecção de inatividade em um intervalo de três segundos. Ao identificar a falha, o sistema removia o sensor da lista ativa e acionava um estado seguro (*fail-safe*) no atuador correspondente, evitando decisões baseadas em dados inválidos.

Outra limitação observada foi a necessidade de sincronização adequada entre as *threads* de comunicação e a interface gráfica JavaFX. Tentativas iniciais de atualização direta da interface a partir de *threads* secundárias resultaram em exceções de execução. Esse problema foi solucionado com o uso do método `Platform.runLater()`, que garante que as atualizações sejam realizadas na *thread* principal da interface gráfica.

De modo geral, os resultados indicam que a solução proposta atende aos requisitos de desempenho e confiabilidade estabelecidos, embora evidencie desafios típicos de sistemas distribuídos, especialmente no tratamento de falhas e na sincronização entre componentes concorrentes.

5. Conclusão

O desenvolvimento deste projeto permitiu a aplicação prática de conceitos fundamentais de redes de computadores, sistemas distribuídos e programação concorrente na construção de um sistema de controle inspirado em um mecanismo real de frenagem antibloqueio (ABS). A adoção de uma arquitetura modular distribuída mostrou-se eficaz na redução do acoplamento entre os componentes, além de proporcionar maior flexibilidade e escalabilidade ao sistema.

A utilização de uma abordagem híbrida de comunicação, combinando os protocolos UDP e TCP, revelou-se adequada aos diferentes requisitos da aplicação. Enquanto o UDP atendeu às necessidades de baixa latência na transmissão de telemetria, o TCP garantiu a confiabilidade necessária para o envio de comandos críticos. Os resultados experimentais confirmaram que essa estratégia equilibra desempenho e robustez de forma satisfatória.

Do ponto de vista de implementação, o projeto evidenciou desafios relevantes associados à concorrência e à sincronização entre *threads*, especialmente na integração com interfaces gráficas. A necessidade de utilizar mecanismos como `CountDownLatch` e `Platform.runLater()` reforçou a importância de compreender o modelo de execução da linguagem e as limitações impostas por diferentes camadas do sistema.

Além disso, aspectos relacionados à infraestrutura, como a tentativa de utilização de contêineres Docker, evidenciaram limitações práticas que nem sempre são abordadas em nível teórico. A dificuldade de execução de aplicações JavaFX em ambientes conteinerizados demonstrou a importância de considerar restrições do ambiente de execução desde as fases iniciais do projeto.

No âmbito do aprendizado, o problema proposto contribuiu significativamente para a consolidação de conhecimentos teóricos por meio de sua aplicação prática. A integração entre comunicação em rede, processamento concorrente e lógica de controle proporcionou uma visão mais abrangente sobre o desenvolvimento de sistemas distribuídos reais.

Como crítica construtiva, destaca-se que, embora a liberdade na escolha do sistema a ser desenvolvido seja um aspecto positivo, a forma como o problema foi inicialmente proposto pode dificultar a compreensão do escopo completo da atividade, especialmente por se tratar do primeiro projeto da disciplina. A ausência de um direcionamento mais estruturado sobre os requisitos globais e cenários esperados ao longo do desenvolvimento tornou mais desafiador antecipar as etapas necessárias para a implementação completa da solução. Uma definição inicial mais clara do panorama geral poderia auxiliar no planejamento e na tomada de decisões técnicas desde as fases iniciais do projeto.

De forma geral, o projeto foi relevante e desafiador, contribuindo para o desenvolvimento de habilidades técnicas e analíticas importantes para a formação acadêmica e profissional, especialmente em áreas relacionadas a sistemas distribuídos, desenvolvimento de software e engenharia de sistemas.

Referências

Atzori, L., Iera, A., and Morabito, G. (2010). The internet of things: A survey. *Computer Networks*, 54(15):2787–2805.

Coulouris, G., Dollimore, J., and Kindberg, T. (2005). *Distributed Systems: Concepts and Design*. Addison-Wesley, 4th edition.

de Oliveira, T., Rodrigues, J. J. P. C., Kozlov, S. A., and Rabêlo, R. A. L. (2014). Internet of things: A survey of technologies and applications. *Journal of Information Systems and Technology Management*, 11(2):261–274.

Inc., D. (2025). Docker documentation. `https://docs.docker.com/`. Acesso em: março 2026.

Oracle (2024). Java documentation - networking and concurrency. `https://docs.oracle.com/javase/tutorial/`. Acesso em: março 2026.

Tanenbaum, A. S. and Wetherall, D. J. (2011). *Computer Networks*. Pearson, 5th edition.

A. Interfaces do Sistema

Figura 2. *
(a) Sensores

Figura 3. *
(b) Atuadores

Figura 4. *
(c) Usuário

Figura 5. *
(d) Servidor

Figura 6. Interfaces do sistema: sensores, atuadores, usuário e servidor